

NeoCis Assessment Report

Ojas Taskar
otaskar1@jh.edu

May 3, 2025

1 Forward and Inverse Kinematics

1.1 Explanation

Approach: We reduce the complexity of three-link inverse kinematics by converting it to a two link problem. Given input coordinates by mouse click, we calculate the corresponding desired location of point C (Fig-1).

Later, by solving for theta1 and theta2, the joint angles nearer to base, we figure out theta3 (the joint angle of the link CD) by using simple trigonometric relations. In the inverse kinematics, we notice that two unique solutions are possible for theta2 (Fig-2). We choose the one that minimizes max angle delta in regular planner and the angles for each step over the entire dynamic graph in dynamic approach.

For forward kinematics, we use simple 2D link-to-link forward kinematics calculations and compose them together as SE(2) to get the final forward kinematics transform. I pre-computed these configurations using MATLAB for computation and converted them into MATLAB code.

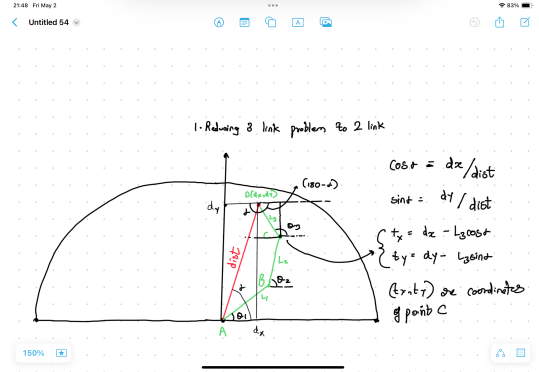


Figure 1: Inverse Kinematics Calculation-1

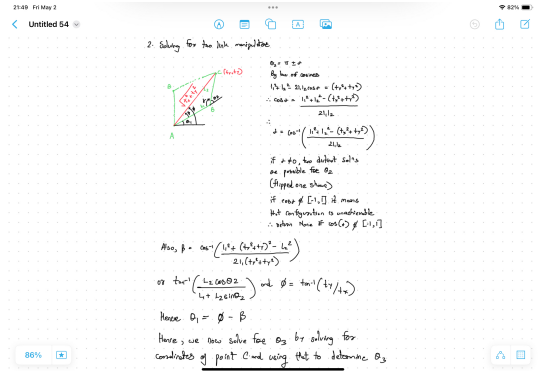


Figure 2: Inverse Kinematics Calculation-2

1.2 Technical Implementation

Approach: I used tkinter due to previous ease of use and approach straightforward-ness. The input clicks are pretty easy to bind using tkinter. The on-click function contains the necessary documentation for the entire application. For the extra credit of minimizing max delta over all possible steps, I used a dynamic programming approach to define a graph structure where each node represents an angle solution for a particular one of the total 21 points. Using iteration over the graph with the edge weights representing the 'max delta' over that transition, I used a backtracking framework to output best joint configuration over the search space. I chose DP since 2^{21} possible combinations exist for joint selection over all the joint spaces, making normal search intractable. However, this is a work in progress.